UNIVERSIDADE CATÓLICA DE BRASÍLIA
Bacharelado em Engenharia de Software

RELATÓRIO DA ETAPA 2

Sistema de Gestão para Rede de Cinemas

Laboratório de Banco de Dados · GPE17M40083
Professor: Samuel Novais Moura Júnior
Ano/semestre: 2026/2

Arthur Moreira Alves da Silva · Andrey Gabriel Lins Ferreira
Paulo Fernando Santos Amorim dos Reis · Alexandre Braga Rodrigues
Ana Beatriz Caldeira Faulin

Panorama do projeto

O repositório contém uma aplicação web conectada ao MySQL, cadastros, venda de ingressos e produtos e consultas com filtros. A checagem do código também identificou requisitos da Etapa 2 que ainda precisam de implementação ou evidência prática.

Estado verificado no repositório

Área	O que existe	O que falta confirmar ou corrigir
Aplicação	Seis cadastros e fluxos de venda; consultas parametrizadas por período e filtro.	A view vw_relatorio_sessao chamada pela consulta de ocupação não aparece no DDL. Revisar também o fallback de mensagens, que pode devolver texto bruto da exceção.
SQL e senhas	Valores de consulta vinculados por parâmetros JDBC; senha com PBKDF2 e sal aleatório.	Executar e registrar o teste HTTP de injeção SQL com a aplicação ligada.
Transações	@Transactional nas vendas; bloqueio de produto/sessão com FOR UPDATE.	Demonstrar sucesso, falha com rollback e disputa simultânea de duas sessões.
Desempenho	Índices definidos no DDL para sessão, ingresso, venda e fidelidade.	Medir três consultas com EXPLAIN antes/depois, tempos e volume de dados.
Banco e operação	Quatro gatilhos validam regras do domínio; configuração aceita variáveis externas.	Criar perfis SQL, auditoria, plano de restauração e evidências correspondentes.

Ordem sugerida para fechar a entrega

1. Criar ou corrigir a view do relatório e validar a tela no banco da equipe.
2. Registrar os testes de transação/rollback, concorrência e SQL injection.
3. Executar os três estudos de desempenho e anexar planos e medições.
4. Implementar perfis SQL e auditoria; documentar o plano de backup e testar uma restauração.
5. Revisar os artefatos da Etapa 1 e preparar a apresentação ao vivo de 15 minutos.

A lista completa de critérios da Etapa 2 aparece no anexo final. Os itens parciais ou pendentes não são apresentados como concluídos.

1. Objetivo e escopo

Este relatório descreve a aplicação construída para operar o banco de dados do cinema e confronta o estado atual do projeto com os itens B1 a B8 da Etapa 2. A aplicação foi desenvolvida em Java com Spring Boot, Thymeleaf e acesso JDBC ao MySQL. O banco usa tabelas InnoDB, chaves estrangeiras, restrições e gatilhos de validação de regras de negócio.

O sistema cobre cadastros de clientes, funcionários, filmes, salas, produtos e sessões; venda de ingressos com escolha de assento; venda de produtos; consulta de vendas e controle de acesso por perfil de funcionário. O DDL declara 19 tabelas para filmes, salas, assentos, clientes, funcionários, sessões, ingressos, vendas, pagamentos e fidelidade.

A implementação deve ser entendida como uma fotografia do repositório, não como prova de que todos os experimentos da disciplina já foram executados. Quando não há registro ou arquivo de evidência, o item aparece como pendente no checklist.

2. Aplicação e consultas (B1 e B2)

A área gerencial oferece formulários de inclusão, edição, listagem e exclusão para seis cadastros: clientes, funcionários, filmes, salas, produtos e sessões. Na área do funcionário, a tela de bilheteria mostra sessões por filme, apresenta informações do filme e permite continuar para a escolha de assento. As vendas de produtos e ingressos são fluxos separados.

As consultas de ingressos vendidos filtram por período e filme e agrupam os resultados por sessão. A consulta de produtos vendidos filtra por período e categoria e agrega unidades e faturamento. O código usa JdbcTemplate com parâmetros para os valores recebidos dos formulários. O gráfico geral de vendas reúne ingressos e produtos por período.

Pendência funcional encontrada: o método de relatório executa `SELECT * FROM vw_relatorio_sessao`, mas não foi encontrada a criação dessa view em `sql/01_ddl.sql` nem em outro SQL do repositório. A tela “Ocupação e receita” pode falhar até que a view seja criada ou a consulta seja ajustada. O checklist registra esse ponto.

Na revisão estática, os dados das consultas são enviados por placeholders. A montagem dinâmica em CadastroRepository usa tabela e colunas definidas pelo catálogo de cadastros do próprio servidor; os dados do formulário continuam parametrizados. Não foi identificada concatenação de valores fornecidos pelo usuário dentro das consultas SQL.

A aplicação lê a configuração do banco por variáveis como CINEMA_DB_URL, CINEMA_DB_USER e CINEMA_DB_PASSWORD, com suporte local a um arquivo .env. Esse arquivo está listado no .gitignore. Antes de executar em outra máquina, é necessário fornecer as credenciais e confirmar que a conta do banco tenha apenas os privilégios necessários.

3. Teste de injeção SQL e proteção de senhas

Foi feita uma revisão estática dos pontos de acesso ao banco. As entradas de login, filtros de consulta, identificadores de venda e campos dos cadastros são enviados como valores vinculados pelo JDBC, não incorporados ao texto SQL. Um exemplo de entrada de teste é `' OR 1=1 --`; em uma execução real, ela deve ser tratada como texto literal e não autenticar nem ampliar o resultado da consulta.

A tentativa HTTP local não produziu uma resposta porque a aplicação não estava acessível durante a verificação. Portanto, este relatório não declara que o teste dinâmico passou: falta repetir a tentativa com o serviço iniciado, guardar a requisição e a resposta e confirmar que nenhum acesso indevido ocorreu.

As senhas de funcionários são armazenadas com PBKDF2-HMAC-SHA256, 210.000 iterações e sal aleatório de 16 bytes. A validação compara os hashes com MessageDigest.isEqual. O interceptor limita as rotas gerenciais a usuários de perfil gerencial e impede que um gerente acesse as rotas operacionais de funcionário. Esses controles

são da aplicação; não substituem os perfis e privilégios de banco exigidos em B5.

Dados pessoais tratados pelo sistema incluem CPF, nome, telefone, e-mail, data de nascimento, login e hash de senha. A carga deve usar dados fictícios. O acesso a cadastros deve ser restrito por função, e arquivos de configuração com credenciais não devem ser enviados ao repositório.

4. Vendas, transações e concorrência (B3)

Venda de produto usa uma transação anotada com @Transactional. O serviço bloqueia a linha do produto com `FOR UPDATE`, verifica e baixa o estoque e grava a venda, o item e o pagamento; quando há cliente, também registra pontos de fidelidade. A venda de ingresso também usa @Transactional e bloqueia a sessão antes de validar assento, preço e documento; depois grava ingresso, histórico de status, venda, vínculo, pagamento e pontos.

Esse fluxo mostra intenção transacional e operações que atingem mais de uma tabela. Entretanto, o repositório não contém evidência de uma falha provocada no meio da venda seguida de conferência do rollback. Também não há relatório do experimento de concorrência com duas sessões simultâneas. O nível de isolamento não é configurado explicitamente pela aplicação; a equipe precisa consultar o valor efetivo no MySQL durante o experimento e justificar a decisão.

Para a demonstração, o roteiro deve registrar: estado inicial das linhas; venda que termina em COMMIT; nova tentativa com falha depois de uma gravação; estado final após ROLLBACK; e comportamento de duas sessões disputando o mesmo produto ou ingresso. Não se deve afirmar bloqueio, espera ou isolamento sem medir no banco usado na apresentação.

5. Desempenho (B4)

O DDL já cria índices para consultas de sessão, assentos, ingressos, vendas e pontos de fidelidade. Entre eles estão índices de data e filme da sessão, assento por sessão, ingresso por cliente, venda por cliente e funcionário. A presença de um índice, por si só, não comprova ganho de desempenho.

Não há arquivo de experimento com três consultas, EXPLAIN antes/depois, tempo medido e volume da base. Essa parte precisa ser executada no MySQL que será usado na demonstração. As consultas de vendas filtram por `DATE(data_compra)` ou `DATE(data_pagamento)`; como a função é aplicada à coluna, a equipe deve verificar com EXPLAIN se isso impede o uso eficiente de um índice de data e considerar intervalos semiabertos como alternativa, medindo os dois planos.

6. Segurança do banco, auditoria e projeto físico (B5 a B7)

O DDL contém quatro gatilhos de validação de regras de negócio: relação entre filmes, conflito de horário, regras do ingresso e limite para cancelamento. Eles não são gatilhos de auditoria: não registram autor e valores anterior/posterior em uma tabela de auditoria. Também não foram encontradas instruções CREATE USER/ROLE, GRANT ou REVOKE, nem uma view de restrição de colunas ou linhas. Os perfis gerente/funcionário existentes são perfis da aplicação, não perfis SQL.

O banco usa InnoDB, tipos numéricos para valores monetários e tipos de data/hora para registros temporais. Uma revisão física deve conferir comprimentos e domínios com o uso real, e manter preços e pagamentos em DECIMAL. Os arquivos SQL atuais não trazem estimativa de crescimento nem procedimento de backup e restauração testada.

Estimativa ilustrativa, não medida: para dimensionamento inicial, supondo 250 ingressos vendidos por dia durante 730 dias, seriam cerca de 182.500 linhas em ingresso e venda_ingresso, além de pelo menos 365.000 mudanças iniciais de status se cada ingresso registrar “Reservado” e “Pago”. Supondo ainda 120 vendas de produtos por dia com uma linha de item por venda, seriam cerca de 87.600 linhas em venda_produto. Esses números são hipóteses para planejamento; devem ser substituídos por volume observado ou cenário acordado pela equipe.

Plano proposto: manter cópia lógica diária e cópia completa semanal, protegidas contra acesso não autorizado; reter cópias diárias por 7 dias e semanais por 4 semanas; guardar pelo menos uma cópia fora da máquina que executa o banco; e testar restauração mensal em ambiente isolado, registrando duração e validação das tabelas. O plano ainda precisa ser configurado e testado. Particionamento não se justifica sem medições; a decisão deve ser reavaliada se o crescimento ou o tempo de consulta mostrar necessidade.

7. Fechamento e uso de IA

O código já contém uma base funcional para cadastros, vendas, consultas e controle de acesso na aplicação. Para a entrega da Etapa 2, ainda faltam itens de evidência prática e alguns componentes do banco exigidos pelo roteiro, em especial a view do relatório consolidado, os perfis SQL, a auditoria, os experimentos de transação/concorrência e os estudos EXPLAIN. O checklist nas páginas finais pode ser usado para dividir essas tarefas.

Declaração de uso de IA: foi utilizado um assistente de IA como apoio à leitura do repositório, à organização deste relatório e à identificação de pendências. A revisão do conteúdo, a execução dos experimentos e a confirmação de que as conclusões correspondem ao ambiente da equipe devem ser feitas pelos integrantes antes da entrega.

Este arquivo descreve a Etapa 2 e não substitui a revisão dos artefatos A1 a A5 da Etapa 1. O roteiro B8 pede um PDF final consolidado com a Etapa 1 revisada; portanto, esse material e os diagramas devem ser incorporados após a equipe confirmar a revisão que deseja entregar.

Anexo — Checklist de entrega da Etapa 2

Situação baseada nos arquivos versionados. “Presente no código” não significa que o teste de execução foi realizado. Atualize o status e anexe evidências depois dos testes da equipe.

Status	Verificação	Situação observada
PARCIAL	A aplicação inicia em máquina limpa seguindo o README.	README ainda não descreve configuração e inicialização completas; não há teste em máquina limpa.
PARCIAL	Há quatro CRUDs, duas consultas parametrizadas e o relatório alimentado por view.	Seis cadastros e duas consultas aparecem no código; a view vw_relatorio_sessao não foi encontrada no DDL.
OK	Nenhuma consulta concatena valores recebidos do usuário.	Revisão estática: valores de formulário passam por parâmetros JDBC; identificadores dinâmicos vêm do catálogo fixo.
OK	As credenciais do banco não estão versionadas.	.env está no .gitignore; application.properties usa variáveis de ambiente. Conferir git status antes de publicar.
PARCIAL	A transação foi testada em sucesso e falha.	@Transactional está implementado; faltam falha induzida e evidência de rollback.
PENDENTE	O experimento de concorrência está documentado.	Não foi encontrado registro de duas sessões simultâneas.
PENDENTE	Há três estudos EXPLAIN com planos, índices, tempos e volume.	Não foi encontrado arquivo de experimentos ou medições antes/depois.
PENDENTE	Três perfis de banco foram criados e testados com login real.	Há controle gerente/funcionário na aplicação; não foram encontradas roles SQL, GRANT ou REVOKE.
OK	As senhas da aplicação usam hash com sal.	Implementação PBKDF2-HMAC-SHA256 com sal aleatório.
PENDENTE	Dois gatilhos de auditoria registram autor, data e valores anterior/posterior.	Os quatro gatilhos encontrados validam regras de negócio; não foi encontrada tabela/gatilho de auditoria.
PARCIAL	O plano de backup indica frequência, retenção e teste de restauração.	Este relatório apresenta uma proposta; ela não está configurada nem testada.
PENDENTE	As correções da devolutiva da Etapa 1 foram incorporadas.	Não confirmado; reconciliar os artefatos A1–A5 antes de fechar a consolidação B8.
PENDENTE	O ambiente da demonstração está pronto, com banco povoado e duas sessões abertas.	Preparação ao vivo não verificável pelo repositório.
PARCIAL	A declaração de uso de IA está preenchida.	Incluída neste relatório; integrantes devem revisar e confirmar a descrição antes de entregar.

Legenda: OK = verificado no repositório; PARCIAL = implementado parcialmente ou sem teste/evidência; PENDENTE = falta implementar ou comprovar. Itens de revisão estática devem ser confirmados pela equipe no ambiente de apresentação.

Arquivos consultados: README.md; sql/01_ddl.sql, sql/02_carga.sql, sql/03_consultas.sql; código Java em service, repository, controller e security; application.properties e .gitignore. Critérios: Projeto_Final_Laboratorio_de_Banco_de_Dados_2026-2, itens B1–B8 e Anexo C.